

INTRODUCCIÓN

1. Problema y Dataset

PROCESAMIENTO

2. Pipeline ETL Spark

3. Índice WQI

4. EDA y Mapa

MODELOS

5. ANN (baseline)

6. Reencuadre

7. Random Forest

8. Gradient Boosting

9. RF vs GB

10. Tres modelos

INFRAESTRUCTURA

11. Arquitectura HPC

CIERRE

12. Conclusiones

13. Trabajo Futuro

PROYECTO II — COMPUTACIÓN DE ALTO RENDIMIENTO

Calidad del Agua con Machine Learning

Ríos de la India — Implementación distribuida con PySpark, Keras y Scikit-Learn sobre 534 estaciones de monitoreo.

AUTHOR Juan Camilo Torres Peña

DATASET

CPCB India · 534 estaciones

STACK

Spark · Hadoop · Keras · sklearn

FECHA

Mayo 2026

Calidad del agua en ríos de la India

Predecir la calidad del agua a partir de parámetros fisicoquímicos — tanto como valor continuo (WQI) como categoría de acción directa.

VARIABLES DEL DATASET		
DO	mg/L	Oxígeno Disuelto · discriminativa
pH	—	Potencial de hidrógeno
CONDUCTIVITY	μS/cm	Conductividad eléctrica
BOD	mg/L	Demanda Biológica de O ₂ · discriminativa
NITRATE_N_NITRITE_N	mg/L	Nitratos y nitritos
FECAL_COLIFORM	UFC/100mL	Coliformes fecales · discriminativa

534

Estaciones de monitoreo

CSV

Cargado desde HDFS

hdfs://...waterquality.csv

Spark

ETL distribuido sobre HPC

PySpark · DataFrames + SQL

3 variables discriminativas destacadas — DO, BOD y FECAL_COLIFORM concentran la mayor parte de la información.

INTRODUCCIÓN

1. Problema y Dataset

PROCESAMIENTO

2. Pipeline ETL Spark

3. Índice WQI

4. EDA y Mapa

MODELOS

5. ANN (baseline)

6. Reencuadre

7. Random Forest

8. Gradient Boosting

9. RF vs GB

10. Tres modelos

INFRAESTRUCTURA

11. Arquitectura HPC

CIERRE

12. Conclusiones

13. Trabajo Futuro

03 · PIPELINE ETL CON APACHE SPARK

De HDFS al modelo en una sola sesión Spark

01

SparkSession.read.csv()

Carga distribuida desde HDFS

02

Filtro nulos + cast a Float

Limpieza de tipos con PySpark SQL

03

Cálculo de sub-índices qr*

qrPH · qrDO · qrCOND · qrBOD · qrNN · qrFecal

04

Cálculo de pesos w*

Ponderación bibliográfica del dominio

05

WQI = Σ w_i

Índice compuesto + etiquetado por rangos

06

Visualización + ML

GeoPandas · Keras · Scikit-Learn

SESSION.PY

```
# Levantamiento Spark sobre clúster HPC
configura = SparkConf()
configura.setAppName("Calidad_Agua_Torres")
sparkS = SparkSession.builder \

    .config(conf=configura).getOrCreate()

# Carga distribuida desde HDFS
df00 = sparkS.read \
    .format("csv") \
    .option("header", "true") \
    .load("hdfs://..waterquality.csv")
df00.count() # 534
```

Ventaja Spark: el mismo código escala a millones de registros sin cambios.

INTRODUCCIÓN

1. Problema y Dataset

PROCESAMIENTO

2. Pipeline ETL Spark

3. Índice WQI

4. EDA y Mapa

MODELOS

5. ANN (baseline)

6. Reencuadre

7. Random Forest

8. Gradient Boosting

9. RF vs GB

10. Tres modelos

INFRAESTRUCTURA

11. Arquitectura HPC

CIERRE

12. Conclusiones

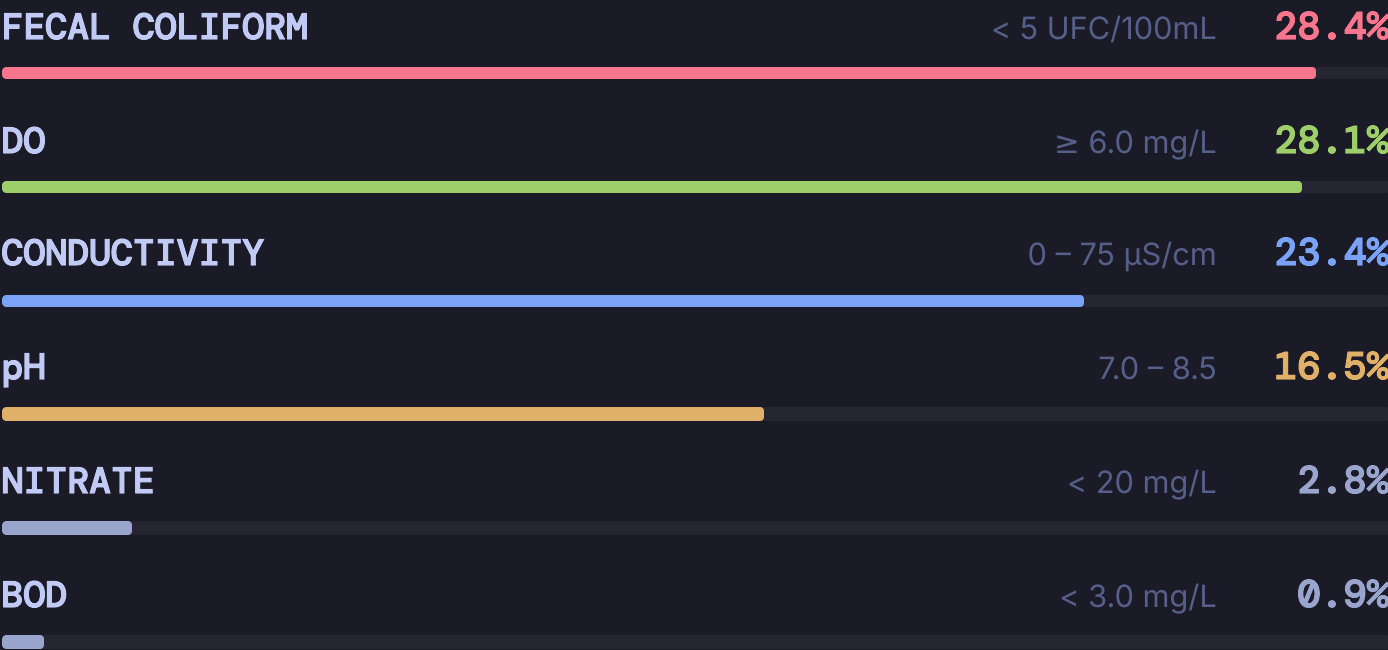
13. Trabajo Futuro

04 · ÍNDICE DE CALIDAD DEL AGUA (WQI)

Sub-índices 0–100, ponderados por literatura científica

Cada parámetro se transforma en un sub-índice qr^* y se pondera con su peso bibliográfico antes de sumarse en el WQI compuesto.

PESOS BIBLIOGRÁFICOS



CATEGORÍAS POR RANGO WQI

● Excelente	[0 , 25)	Apta sin tratamiento
● Buena	[25 , 50)	Tratamiento básico
● Baja	[50 , 75)	Tratamiento estándar
● Muy Baja	[75 , 100)	Tratamiento intensivo
● Inadecuada	≥ 100	Alerta crítica

```
// índice compuesto
WQI = wpH + wDO + wCOND + wBOD + wNN + wFecal
```

INTRODUCCIÓN

1. Problema y Dataset

PROCESAMIENTO

2. Pipeline ETL Spark

3. Índice WQI

4. EDA y Mapa

MODELOS

5. ANN (baseline)

6. Reencuadre

7. Random Forest

8. Gradient Boosting

9. RF vs GB

10. Tres modelos

INFRAESTRUCTURA

11. Arquitectura HPC

CIERRE

12. Conclusiones

13. Trabajo Futuro

05 · ANÁLISIS EXPLORATORIO Y MAPA

Outliers urbanos y un fuerte gradiente Norte–Sur

OUTLIERS DETECTADOS ($|Z| > 3$)

FECAL_COLIFORM ~15%

BOD ~8%

CONDUCTIVITY ~5%

CORRELACIONES CLAVE

BOD ↔ FECAL $r > +0.5$

DO ↔ BOD $r < -0.4$

Desbalanceo de clases detectado → se aplica
`class_weight='balanced'`.

MAPA WQI POR ESTADO (GEOPANDAS)

Norte / Himalaya WQI bajo

Ríos de montaña — poca intervención

Centro WQI medio

Agricultura intensiva — nitratos

Cuencas del Ganges WQI alto

Densidad urbana — contaminación fecal severa

El Ganges en **Varanasi** y **Kanpur** registra los valores más altos de FECAL_COLIFORM del dataset — reflejo directo de densidad poblacional sin saneamiento.

INTRODUCCIÓN

1. Problema y Dataset

PROCESAMIENTO

2. Pipeline ETL Spark

3. Índice WQI

4. EDA y Mapa

MODELOS

5. ANN (baseline)

6. Reencuadre

7. Random Forest

8. Gradient Boosting

9. RF vs GB

10. Tres modelos

INFRAESTRUCTURA

11. Arquitectura HPC

CIERRE

12. Conclusiones

13. Trabajo Futuro

Regresión del WQI · $R^2 = 0.9989$ sobre test

Modelo Keras de regresión continua entrenado sobre los sub-índices qr* para validar el cálculo del WQI.

ARQUITECTURA

Input	[qrPH, qrDO, qrCOND, qrBOD, qrNN, qrFecal]	6
Dense	ReLU	350
Dense	ReLU	350
Dense	ReLU	350
Output	WQI continuo · lineal	1

OPTIMIZER

Adam

LOSS

MSE

TEST

107 muestras

RESULTADOS SOBRE TEST SET

R ²	0.9989	Explica el 99.89% de la varianza
RMSE	0.5137	Error típico ±0.51 puntos WQI
MAE	0.1063	Error absoluto promedio 0.11
Sesgo	0.0381	Prácticamente sin sesgo sistemático

Limitación operacional:

 los sub-índices qr* no existen en campo — solo las mediciones crudas. Esto motiva los modelos RF y GB que clasifican directamente desde sensores.

INTRODUCCIÓN

1. Problema y Dataset

PROCESAMIENTO

2. Pipeline ETL Spark

3. Índice WQI

4. EDA y Mapa

MODELOS

5. ANN (baseline)

6. Reencuadre

7. Random Forest

8. Gradient Boosting

9. RF vs GB

10. Tres modelos

INFRAESTRUCTURA

11. Arquitectura HPC

CIERRE

12. Conclusiones

13. Trabajo Futuro

¿Por qué agregar nuevos modelos?

La ANN funciona — pero requiere sub-índices que no existen en campo. Los nuevos modelos clasifican directamente desde sensores.

ASPECTO	ANN (REGRESIÓN)	RF / GB (CLASIFICACIÓN)
Output	Valor numérico WQI	Categoría de acción directa
Features de entrada	Sub-índices derivados (qr*)	Mediciones crudas directas
Utilidad en campo	Análisis de tendencias	Alertas inmediatas operacionales
Requiere dominio experto	Sí – calcular qr*	No
Acción del operador	Interpretar un número	Leer una etiqueta (Buena/Inadecuada)

MODELO 1

Random Forest

Ensemble paralelo · alta interpretabilidad · robusto a outliers · entrenamiento paralelizable

MODELO 2

Gradient Boosting

Ensemble secuencial · estado del arte en datos tabulares · early stopping automático · mayor precisión

INTRODUCCIÓN

1. Problema y Dataset

PROCESAMIENTO

2. Pipeline ETL Spark

3. Índice WQI

4. EDA y Mapa

MODELOS

5. ANN (baseline)

6. Reencuadre

7. Random Forest

8. Gradient Boosting

9. RF vs GB

10. Tres modelos

INFRAESTRUCTURA

11. Arquitectura HPC

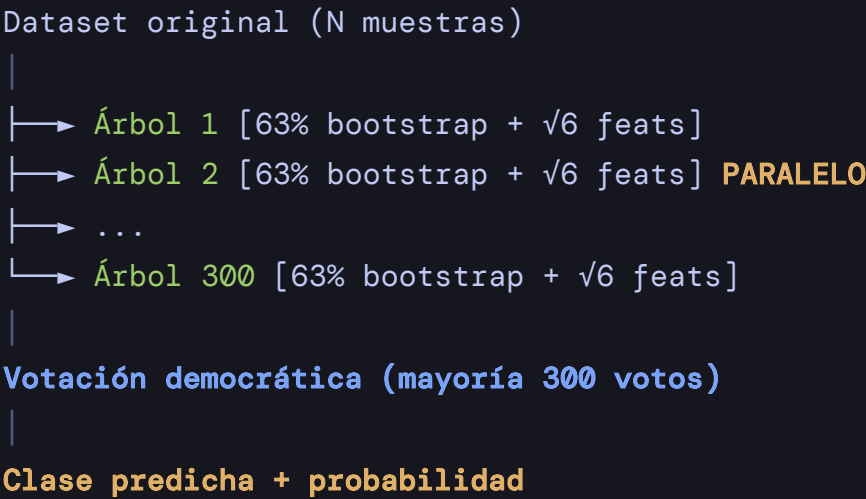
CIERRE

12. Conclusiones

13. Trabajo Futuro

Dos fuentes de aleatoriedad forzada

ESQUEMA DE ENSEMBLE



BOOTSTRAP

Cada árbol ve ~63% de datos. El 37% restante (Out-of-Bag) es validación gratuita.

FEATURE BAGGING

Cada nodo evalúa solo $\sqrt{6} \approx 2$ features al azar → árboles diversos.

LEY DE LOS GRANDES NÚMEROS

Si cada árbol tiene error $\epsilon < 0.5$ y son independientes, el error del ensemble decrece exponencialmente con B.

Reduce varianza sin aumentar sesgo — antídoto directo al overfitting de árboles individuales.

INTRODUCCIÓN

1. Problema y Dataset

PROCESAMIENTO

2. Pipeline ETL Spark

3. Índice WQI

4. EDA y Mapa

MODELOS

5. ANN (baseline)

6. Reencuadre

7. Random Forest

8. Gradient Boosting

9. RF vs GB

10. Tres modelos

INFRAESTRUCTURA

11. Arquitectura HPC

CIERRE

12. Conclusiones

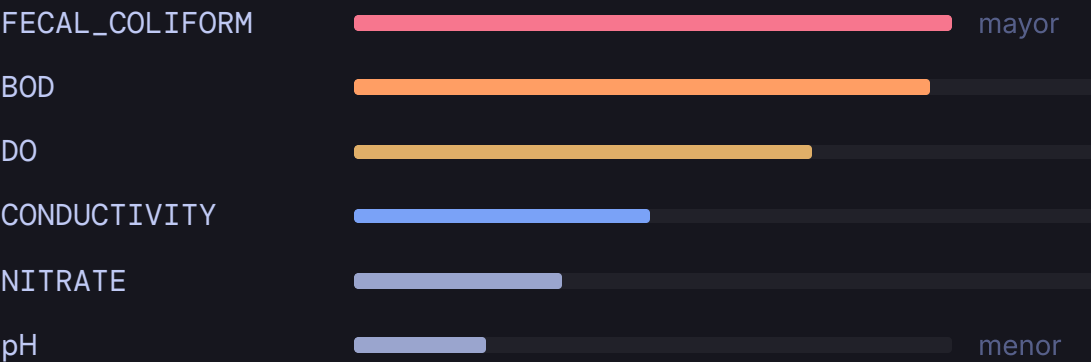
13. Trabajo Futuro

Hiperparámetros y feature importance

HIPERPARÁMETROS CLAVE

n_estimators	300	Estabiliza error; más árboles nunca empeoran
max_depth	12	Reglas complejas — bagging controla overfitting
min_samples_leaf	3	Evita hojas con casos únicos
max_features	'sqrt'	Diversidad entre árboles (Breiman, 2001)
class_weight	'balanced'	Peso inverso a frecuencia — protege minoritarias
n_jobs	-1	Paralelización completa de núcleos

IMPORTANCIA DE FEATURES (GINI)



Ranking coincide con los pesos del WQI — **validación cruzada del conocimiento de dominio.**

INTRODUCCIÓN

1. Problema y Dataset

PROCESAMIENTO

2. Pipeline ETL Spark

3. Índice WQI

4. EDA y Mapa

MODELOS

5. ANN (baseline)

6. Reencuadre

7. Random Forest

3. Gradient Boosting

9. RF vs GB

10. Tres modelos

INFRAESTRUCTURA

11. Arquitectura HPC

CIERRE

12. Conclusiones

13. Trabajo Futuro

Aprendizaje secuencial por corrección de errores

CONSTRUCCIÓN ITERATIVA

```
F0(x) = predicción base (clase más frecuente)
|
▼ ri = yi - p0(xi) ← residuos
Árbol 1 aprende: features → residuos
| F1 = F0 + 0.05 × h1(x)
▼
Árbol 2 aprende los residuos de F1
| F2 = F1 + 0.05 × h2(x)
▼ ...
Árbol t: si val. interna no mejora en 25 rondas
→ EARLY STOP
```

Analogía: aprender a tirar dardos midiendo cuánto fallaste y en qué dirección — cada tiro corrige el error específico del anterior.

DIFERENCIA FILOSÓFICA CON RF

CONSTRUCCIÓN

RF · Paralela · árboles independientes	GB · Secuencial · cada árbol depende del anterior
--	---

ERROR QUE REDUCE

RF · Varianza	GB · Sesgo
----------------------	-------------------

ÁRBOLES

RF · Profundos (max_depth=12)	GB · Superficiales (max_depth=4) — weak learners
-------------------------------	--

REGULARIZACIÓN

RF · Bootstrap + feature bagging	GB · learning_rate + subsample + early stop
----------------------------------	---

INTRODUCCIÓN

1. Problema y Dataset

PROCESAMIENTO

2. Pipeline ETL Spark

3. Índice WQI

4. EDA y Mapa

MODELOS

5. ANN (baseline)

6. Reencuadre

7. Random Forest

8. Gradient Boosting

9. RF vs GB

10. Tres modelos

INFRAESTRUCTURA

11. Arquitectura HPC

CIERRE

12. Conclusiones

13. Trabajo Futuro

Hiperparámetros y ecuación central

HIPERPARÁMETROS CLAVE

n_estimators	300	Límite — early stopping actúa antes
learning_rate	0.05	Shrinkage pequeño: lr bajo + muchos árboles
max_depth	4	Weak learners — GB amplifica errores
min_samples_leaf	5	Más conservador que RF
subsample	0.8	Stochastic GB — árboles menos correlacionados
validation_fraction	0.10	10% reservado para early stopping interno
n_iter_no_change	25	Para si no mejora en 25 rondas

ECUACIÓN CENTRAL

$$F_t(x) = F_{t-1}(x) + \eta \cdot h_t(x)$$

η = learning_rate = 0.05
 h_t = árbol sobre residuos ($y - p_{t-1}$)

EARLY STOPPING

Si activa antes de 300 rondas → **evidencia de convergencia real**, no límite arbitrario.

INTRODUCCIÓN

1. Problema y Dataset

PROCESAMIENTO

2. Pipeline ETL Spark

3. Índice WQI

4. EDA y Mapa

MODELOS

5. ANN (baseline)

6. Reencuadre

7. Random Forest

8. Gradient Boosting

9. RF vs GB

10. Tres modelos

INFRAESTRUCTURA

11. Arquitectura HPC

CIERRE

12. Conclusiones

13. Trabajo Futuro

Análisis técnico comparativo

CRITERIO TÉCNICO	RANDOM FOREST	GRADIENT BOOSTING
Estrategia de ensemble	Bagging (paralelo)	Boosting (secuencial)
Tipo de error que reduce	Varianza	Sesgo
Profundidad de árboles	Alta (max_depth=12)	Baja (max_depth=4) – weak learners
Velocidad de entrenamiento	Rápida (n_jobs=-1)	Más lenta (secuencial)
Sensibilidad a hiperparámetros	Baja – robusto	Media – learning_rate crítico
Regularización automática	No	Sí – early stopping
Desempeño en tabular (literatura)	Muy bueno	Mejor (Fernández-Delgado, 2014)
Importancias de features	Gini (reducción impureza)	Reducción de pérdida – más robusta

Random Forest

Despliegues donde velocidad de entrenamiento y simplicidad operacional son prioritarias. Excelente baseline.

Gradient Boosting

★ PROD

Cuando la precisión es crítica — decisiones de salud pública, alertas de contaminación. Recomendado para producción.

INTRODUCCIÓN

1. Problema y Dataset

PROCESAMIENTO

2. Pipeline ETL Spark

3. Índice WQI

4. EDA y Mapa

MODELOS

5. ANN (baseline)

6. Reencuadre

7. Random Forest

8. Gradient Boosting

9. RF vs GB

0. Tres modelos

INFRAESTRUCTURA

11. Arquitectura HPC

CIERRE

12. Conclusiones

13. Trabajo Futuro

ANN, Random Forest y Gradient Boosting lado a lado

CRITERIO	ANN (KERAS)	RANDOM FOREST	GRADIENT BOOSTING
Tarea	Regresión (WQI)	Clasificación	Clasificación
Features de entrada	Sub-índices qr*	Mediciones crudas	Mediciones crudas
Normalización	Requerida (StandardScaler)	No necesaria	Buena práctica
Outliers	Sensible	Robusto (árboles)	Robusto (árboles)
Clases desbalanceadas	Gestión manual	class_weight='balanced'	class_weight + subsampling
Interpretabilidad	Baja (caja negra)	Alta (feature importances)	Media
Overfitting	Riesgo medio	Controlado por bagging	Early stopping automático
Escalabilidad	Media	Alta (n_jobs=-1)	Media (secuencial)
Requiere dominio experto	Sí	No	No
Desempeño tabular	Bueno	Muy bueno	Mejor
Uso recomendado	Tendencias WQI	Baseline rápido	★ Producción

Los modelos son **complementarios**: ANN para tendencias continuas del WQI · RF/GB para clasificación operacional directa en campo.

INTRODUCCIÓN

1. Problema y Dataset

PROCESAMIENTO

2. Pipeline ETL Spark

3. Índice WQI

4. EDA y Mapa

MODELOS

5. ANN (baseline)

6. Reencuadre

7. Random Forest

8. Gradient Boosting

9. RF vs GB

10. Tres modelos

INFRAESTRUCTURA

1. Arquitectura HPC

CIERRE

12. Conclusiones

13. Trabajo Futuro

¿Por qué esta arquitectura importa en HPC?

CAPA DISTRIBUIDA

HDFS · Apache Spark

- Replicación de datos
- DAG transformaciones
- Tolerancia a fallos
- Procesamiento en memoria
- Particionamiento
- SQL distribuido

`.toPandas()` · nodo driver ▼

CAPA LOCAL

Scikit-Learn · Keras/TensorFlow

- Entrenamiento en driver
- GPU opcional
- Iteración eficiente
- Modelos en memoria

DECISIONES DE DISEÑO HPC

ETL en Spark, ML en sklearn

Spark no es óptimo para iteración de ML — sklearn lo es para ~500 muestras.

HDFS como fuente de datos

Desacoplamiento: datos en clúster, cómputo portable.

n_jobs=-1 en RF

Paralelización perfecta — 300 árboles independientes en todos los núcleos.

`.toPandas()` solo al final

Minimiza transferencia driver ↔ workers — principio de localidad de datos.

INTRODUCCIÓN

1. Problema y Dataset

PROCESAMIENTO

2. Pipeline ETL Spark

3. Índice WQI

4. EDA y Mapa

MODELOS

5. ANN (baseline)

6. Reencuadre

7. Random Forest

8. Gradient Boosting

9. RF vs GB

10. Tres modelos

INFRAESTRUCTURA

11. Arquitectura HPC

CIERRE

2. Conclusiones

13. Trabajo Futuro

Sobre los modelos y la arquitectura HPC

MODELOS DE ML

- Gradient Boosting obtiene el mejor desempeño — recomendado para alertas operacionales.
- Random Forest es la alternativa óptima cuando se prioriza velocidad, interpretabilidad y simplicidad de despliegue.
- ANN complementa con predicción continua del WQI para análisis de tendencias temporales.
- Los tres coinciden en FECAL_COLIFORM, BOD y DO como las variables más informativas — validación empírica del WQI.

ARQUITECTURA HPC

- Spark + HDFS demostró ser la elección correcta — el código escala a millones de registros sin modificaciones.
- La separación ETL distribuido / ML iterativo refleja un principio central del HPC: la herramienta correcta en cada capa.
- La paralelización nativa de RF (n_jobs=-1) ilustra cómo los ensembles son inherentemente paralelizables.
- Validación cruzada estratificada es indispensable con datasets pequeños desbalanceados.

INTRODUCCIÓN

1. Problema y Dataset

PROCESAMIENTO

2. Pipeline ETL Spark

3. Índice WQI

4. EDA y Mapa

MODELOS

5. ANN (baseline)

6. Reencuadre

7. Random Forest

8. Gradient Boosting

9. RF vs GB

10. Tres modelos

INFRAESTRUCTURA

11. Arquitectura HPC

CIERRE

12. Conclusiones

3. Trabajo Futuro

De prototipo a clúster productivo

MEJORA	TÉCNICA PROPUESTA	IMPACTO ESPERADO
Optimización de hiperparámetros	Optuna · BayesSearchCV distribuido	+2-5% F1
Explicabilidad avanzada	SHAP values	Confianza regulatoria y auditoría
Dimensión temporal	Series de tiempo + LSTM	Predicción proactiva de contaminación
Datos geoespaciales	Encoding distancia a ciudades/industrias	Captura patrones regionales
Ensemble final	Stacking (RF+GB+ANN) en el driver	Máximo desempeño posible
Despliegue en tiempo real	FastAPI + Docker + Kafka + Spark Streaming	Alertas IoT en milisegundos
Escala HPC real	Spark MLlib (GBT, RF distribuido)	ETL + entrenamiento en el mismo clúster

El salto natural: migrar el entrenamiento a **Spark MLlib** — RF y GBT están implementados de forma nativa con paralelización distribuida, eliminando el cuello de botella del `.toPandas()` al driver.

INTRODUCCIÓN

1. Problema y Dataset

PROCESAMIENTO

2. Pipeline ETL Spark

3. Índice WQI

4. EDA y Mapa

MODELOS

5. ANN (baseline)

6. Reencuadre

7. Random Forest

8. Gradient Boosting

9. RF vs GB

10. Tres modelos

INFRAESTRUCTURA

11. Arquitectura HPC

CIERRE

12. Conclusiones

13. Trabajo Futuro

CIERRE · RESUMEN EJECUTIVO

Gracias.

Calidad del agua en ríos de la India con pipeline reproducible PySpark + Scikit-Learn + Keras sobre clúster Hadoop HPC.

SUMMARY.JSON

Problema → "Clasificación calidad del agua (5 clases) + regresión WQI"

Dataset → "534 estaciones · India · 6 parámetros"

HPC Stack → "Apache Spark + Hadoop HDFS (ETL distribuido)"

ML Stack → "Keras/TF + Scikit-Learn (entrenamiento en driver)"

Modelo 1 ANN → R²=0.9989 · RMSE=0.5137

Modelo 2 Random Forest → clasificación directa · alta interpretabilidad

Modelo 3 Gradient Boosting → early stopping · mejor desempeño

Recomendado → "Gradient Boosting para producción"

AUTOR

Juan Camilo Torres Peña

Maestría en Computación de Alto Rendimiento — 2026